Manual técnico

Ingreso & Validación Itsanet · Arquitectura, datos, integraciones y mantenimiento

Contenido

1. Identificación y alcance

2. Arquitectura y comunicación

3. Lenguajes, componentes y código fuente

4. Servidor e infraestructura observada

5. Modelo de datos e integridad

6. Inventario funcional de las tablas

7. Ciclo de vida de las imágenes y evidencias

8. Organización de carpetas en Google Drive

9. Autorización Google, correo y catálogo

10. Configuración, seguridad y sincronización

11. Calendario de mantenimiento

12. Respaldo, recuperación y atención de incidentes

13. Consultas de operación y diagnóstico

14. Diccionario técnico del modelo

15. Referencias y conservación de la documentación

1. Identificación y alcance

IV ITSANET (Ingreso & Validación Itsanet) es una aplicación web para recepción y validación física de mercadería, registro de evidencias, trazabilidad por ítem, generación de informes y entrega de correo.

Control	Referencia
Documento	ITSANET-MT-20261029 · revisión 1.2
Fecha	29 de octubre de 2026 · America/Guayaquil
URL operativa	https://apps.flexnetuio.com:8443/app/
Código funcional documentado	v69 · recursos 20261024-69
Commit funcional	`cda9187ea125aa710fdd29c093ddf2c16f2bc5fa`
Estado operativo	En línea, confirmado por el responsable el 28/10/2026.

El manual combina revisión estática del código de referencia y salidas de terminal del servidor compartidas el 27/10/2026. El inventario de tablas describe el modelo declarado; no es una extracción del esquema productivo. La revisión Git instalada, el nombre del esquema, las versiones efectivas de Python/Apache y los parámetros privados se cotejan en el inventario de operación.

2. Arquitectura y comunicación

01 · ACCESO

PC / handheld

Navegador · HTTPS · /app/

02 · ENTRADA

Apache2 · 8443

Receptor del puerto publicado

03 · APLICACIÓN

Uvicorn · 3002

FastAPI · reglas y API

04 · DATOS

MySQL

SQLAlchemy + PyMySQL · transacciones

El navegador carga HTML, CSS y JavaScript bajo `/app/`. Las solicitudes de registro y consulta llegan al backend por la publicación HTTPS de Apache. Uvicorn ejecuta la aplicación ASGI de FastAPI; esta verifica sesión, permisos y reglas y utiliza SQLAlchemy/PyMySQL para persistir en MySQL.

Enlace	Protocolo / destino	Responsabilidad
PC / handheld → Apache	HTTPS · apps.flexnetuio.com:8443	Certificado, publicación y acceso web.
Apache → Uvicorn	Backend en puerto 3002	Enrutamiento hacia interfaz y API; conservar esquema HTTPS y transmisión SSE.
FastAPI → MySQL	SQLAlchemy + PyMySQL	Transacciones, consultas, integridad y colas durables.
Backend → Google	HTTPS · OAuth, Drive API y Gmail API	Autorización, copia de evidencias y entrega del correo.
Backend → Depot	API HTTPS, si está habilitada	Consulta del catálogo/stock y caché local.

Las directivas efectivas de proxy, el certificado y sus vencimientos se administran en Apache. La escucha local de un puerto no demuestra su accesibilidad desde Internet.

3. Lenguajes, componentes y código fuente

Capa	Tecnología	Función
Backend	Python	Reglas de negocio, persistencia, permisos e integraciones.
API	FastAPI 0.143.0 + Pydantic 2.13.5	Rutas HTTP, validación de solicitudes y respuestas.
Servidor de aplicación	Uvicorn 0.53.0	Servidor ASGI; no sustituye la publicación Apache.
Acceso a datos	SQLAlchemy 2.1.4 + PyMySQL 1.1.1	ORM y conector del motor MySQL.
Interfaz	HTML5, CSS y JavaScript modular	Pantallas de PC/handheld, captura, validación y actualización de vistas.
Archivos	SheetJS, openpyxl, Pillow, jsPDF	Excel, imágenes, documentos y etiquetas.
Códigos de barras	JsBarcode / ZXing	Generación CODE128 y lectura compatible.
Construcción y pruebas	Node.js, esbuild, pytest	Preparación de recursos y verificación; Node no es el backend productivo de esta aplicación.

Las versiones anteriores son dependencias declaradas en el repositorio; no se presentan como una lectura de paquetes instalados en producción. SQLite, Firebase y Apps Script permanecen como opciones de desarrollo/compatibilidad; la arquitectura corporativa documentada usa Apache y MySQL.

```
IV_ITSANET/
├── app/main.py              # FastAPI, arranque y recursos /app/
├── app/config.py            # Parámetros y rutas
├── app/db.py                # Motor, sesiones y ajustes SQL
├── app/models.py            # Modelo relacional
├── app/routers/             # Endpoints y permisos
├── app/services/            # Operación, archivos y Google/Depot
├── firebase/public/         # HTML, CSS, módulos JS y service worker
├── scripts/                 # Actualización, respaldo y diagnóstico
├── tests/                   # Verificaciones aisladas
├── requirements.txt         # Dependencias Python
└── package.json             # Herramientas y dependencias JS
```

4. Servidor e infraestructura observada

Elemento	Valor / situación observada
Host / IP local	apps · 192.168.4.94 · interfaz ens160
MySQL local	8.0.46-0ubuntu0.24.04.4 · comentario Ubuntu
Servicio de aplicación	iv-itsanet.service · Uvicorn en 0.0.0.0:3002
Publicación	Apache2 en *:8443 · itsanet-app-8443.conf
Archivo del virtual host	/etc/apache2/sites-enabled/itsanet-app-8443.conf
MySQL	127.0.0.1:3306; listener adicional 127.0.0.1:33060
Recursos, captura del 27/10	4 CPU lógicas; 11 GiB RAM, 9,6 GiB disponibles; raíz 73 GB, 50 GB disponibles.
Servicios compartidos	Apache 80/443 para Data Sync; contenedor itsanet-data-sync saludable, 127.0.0.1:8080.
Otros servicios	SSH 22; pm2-root.service habilitado pero inactivo/dead en la consulta.
Filtrado	UFW inactivo; existen reglas nftables gestionadas por Docker.

El resultado de versión MySQL identifica la instancia local consultada. La conexión efectiva de la aplicación se define en `VAL_DATABASE_URL`; el esquema y usuario se mantienen en configuración protegida. El valor de capacidad es una medición puntual, no una garantía de capacidad futura.

El servidor es compartido: las actualizaciones del aplicativo deben conservar MySQL, los otros virtual hosts y el contenedor Data Sync. Revisar la exposición de 3002 conforme a la política de red, dado que el binding observado es 0.0.0.0.

5. Modelo de datos e integridad

El modelo declara **30 tablas**. Las tablas `app_*` conservan la operación web actual; las tablas iniciales de recepción coexisten para compatibilidad. `operadores`, `sesiones` y el maestro de productos son componentes compartidos. No eliminar tablas de compatibilidad sin revisar sus consumidores y migración.

```
app_recepciones.codigo
├── app_lineas             (packing original y orden)
├── app_skus               (estado agregado por SKU)
│   └── app_calidad_skus
├── app_eventos            (libro de registros y correcciones)
├── app_unidades_logisticas (bultos y cajas)
├── app_archivos
│   ├── app_archivo_identidades
│   ├── app_archivo_skus
│   ├── app_evidencias     (SKU, categoría y unidad)
│   └── app_copias_drive
├── app_informes / app_entregas
└── app_presencias / app_ingresos / app_impresiones

operadores → sesiones y presencias
productos → producto_codigos
app_stock_depot: clave compuesta SKU + cliente
```

- InnoDB y utf8mb4 definidos para las tablas MySQL; claves primarias, claves externas e índices según el modelo.
- Cantidades: DECIMAL(14,3); precios: DECIMAL(14,4). SQLAlchemy devuelve estos valores a Python como float según la configuración declarada; deben conservarse las validaciones de suma y exportación.
- Fechas: DATETIME con milisegundos en MySQL. La sesión SQL utiliza UTC; la presentación operativa usa America/Guayaquil.
- Modo SQL estricto: STRICT_ALL_TABLES y NO_ENGINE_SUBSTITUTION. Pool con pre-ping; valores predeterminados: tamaño 5, overflow 5, reciclaje 3600 s.
- El identificador del evento permite reconocer reintentos. Las correcciones conservan motivo y registro compensatorio; app_skus es una proyección recalculable de eventos.
- Las relaciones CASCADE se documentan en el diccionario: una eliminación de cabecera puede afectar hijos. Las intervenciones directas en SQL requieren revisión y respaldo.

El árbol resume relaciones funcionales; el diccionario indica cuáles están declaradas como claves externas. La bitácora de aplicación no equivale a almacenamiento inmutable frente a un administrador de base de datos.

6. Inventario funcional de las tablas

Tabla	Responsabilidad
`operadores`	Identidad, hash de credencial, rol, habilitación y firma de registros.
`sesiones`	Sesiones locales y vencimiento; referencia al operador.
`productos`	Maestro de productos y estado de verificación del catálogo.
`producto_codigos`	Códigos alternativos asociados al SKU del maestro.
`recepciones`	Cabecera del modelo inicial de recepción; compatibilidad.
`packing_items`	Líneas del packing del modelo inicial; compatibilidad.
`pallets`	Pallets del modelo inicial; compatibilidad.
`scan_events`	Eventos del modelo inicial; compatibilidad.
`skus_pendientes`	Productos por resolver del modelo inicial; compatibilidad.
`auditoria`	Auditoría del modelo inicial; compatibilidad.
`app_recepciones`	Cabecera operativa, cliente, bultos, estado, fechas, orígenes y embarques.
`app_lineas`	Líneas originales del packing, orden, SKU, cantidad y columnas adicionales.
`app_skus`	Proyección de esperado/contado y distribución por pallet y origen.
`app_calidad_skus`	Clasificación y datos de calidad relacionados con el SKU operativo.
`app_presencias`	Operador/dispositivo activo y última actividad dentro de una recepción.
`app_informes`	Versiones de informe, tipo, huella de datos, estado y autoría.
`app_auditoria_general`	Acciones generales, referencias y datos de auditoría del aplicativo.
`app_eventos`	Libro de eventos operativos: conteos, cero, anulaciones y motivos.
`app_huellas`	Huellas del packing para reconocer contenido previamente cargado.
`app_archivos`	Metadatos de archivos, SHA-256, tamaño y referencias confirmadas de Drive.

Tabla	Responsabilidad
`app_archivo_identidades`	Secuencia y procedencia de fotografías, independientes del ID remoto.
`app_archivo_skus`	Relación entre un archivo y uno o varios SKU sin duplicar su contenido.
`app_config`	Configuraciones, reservas de identidad y credenciales de integración cifradas.
`app_entregas`	Cola durable de entregas, instantánea de datos, intentos y resultado.
`app_ingresos`	Datos complementarios del ingreso asociados a la recepción.
`app_unidades_logisticas`	Bultos/cajas y relación jerárquica padre-hijo.
`app_evidencias`	Vínculo archivo-recepción-SKU-unidad-categoría y eliminación lógica.
`app_copias_drive`	Cola y estado de copia a Drive, intentos, próximo intento y error.
`app_stock_depot`	Caché de posiciones de stock por SKU y cliente.
`app_impresiones`	Trabajos de impresión, estación, reserva y seguimiento de estado.

Las extensiones JSON conservan mapas, columnas adicionales y estados que no se fijan en columnas escalares. El manejo de estos contenidos se realiza mediante servicios de la aplicación, preservando compatibilidad y versiones de sus estructuras.

7. Ciclo de vida de las imágenes y evidencias

Los bytes del archivo se almacenan fuera de MySQL. La base conserva identidad, tamaño, SHA-256, autoría, clasificación, vínculos al registro y estado de copia. Una misma imagen puede justificar varios SKU mediante asociaciones y accesos directos, sin crear un archivo físico por cada vínculo.

Paso	Procesamiento	Registro principal
1. Captura	Cámara/galería; selección de recepción, embarque, SKU y categoría.	Borrador del navegador cuando es posible.
2. Guardado	Archivo operativo y metadatos; vinculación con bulto/SKU/incidencia.	app_archivos, app_evidencias y asociaciones.
3. Programación	Persistencia de la carga pendiente y del destino.	app_copias_drive.
4. Transferencia	OAuth; reserva de ID remoto; carga multipart; comparación MD5 remoto con contenido.	Drive API + datos de la cola.
5. Confirmación	Guardar ID, enlace y destino comprobado. Conservar miniatura.	drive_id / drive_url y estado COPIADA.
6. Recuperación	Reintento con identidad estable; errores de cuenta/permisos pasan a revisión.	Intentos, próximo intento, error y monitor.

Estados de copia: `PENDIENTE`, `PROCESANDO`, `COPIADA`, `REVISION` y `ELIMINADA`. Una miniatura visible no confirma la copia a Drive. La aplicación comprueba además la ubicación vigente; mover o renombrar carpetas fuera del flujo puede producir una revisión del destino.

```
VAL_DATA_DIR/
├── app_archivos/ID_ARCHIVO     # contenido local retenido
├── app_miniaturas/ID_ARCHIVO   # vista previa
├── secretos/integraciones.key # clave de cifrado, acceso restringido
├── uploads/                   # archivos de entrada según flujo
├── exports/                   # salidas según flujo
└── fotos/                     # rutas de compatibilidad
```

Tras confirmar Drive puede liberarse el original local y conservarse su miniatura. La recuperación de evidencias requiere una política propia de Drive, además del respaldo del servidor. El archivo de configuración protegido y la clave de cifrado forman parte del respaldo seguro.

8. Organización de carpetas en Google Drive

```
CARPETA_RAIZ_CONFIGURADA/
└── CLIENTE/                        (si está definido)
    └── MARCA/                      (si está definida)
        └── CODIGO_RECEPCION/
            ├── INFORMES/           (al subir un informe)
            └── EMBARQUE/           (cuando la evidencia tiene embarque)
                ├── PRE-INGRESO/
                ├── PANORAMICAS/
                ├── FOTOS-FINALES/
                ├── BULTOS/
                │   └── BultoNN_recepcion_embarques/
                │       └── CajaNN_codigo/       (si aplica)
                ├── PRODUCTOS/SKU/
                ├── PNC/SKU/
                ├── SOBRANTES/SKU/
                └── FALTANTES/SKU/
```

El árbol muestra rutas posibles, no carpetas creadas de forma anticipada. **La preparación consulta las rutas existentes; la subida materializa únicamente la ruta utilizada.** Si una carga falla después de crear su ruta, puede quedar una carpeta sin contenido; no existe una garantía transaccional entre MySQL y Drive ni una limpieza automática de todas las carpetas históricas.

- Cliente y marca son segmentos opcionales. También puede reutilizarse una carpeta de recepción ya identificada y validada.
- Si no hay embarques definidos, la categoría queda directamente bajo la recepción. Con embarques, la evidencia común puede quedar en COMPARTIDAS; la evidencia que requiere asignación puede quedar en POR-ASIGNAR.
- Las rutas de SKU pueden incluir BULTOS/Bulto…/Caja… cuando existe vínculo con una unidad logística. OTRAS-EVIDENCIAS cubre los tipos sin categoría específica.
- INFORMES se ubica directamente bajo la recepción en el servicio de Google Drive API.
- Cuando una imagen documenta varios destinos, se crean accesos directos al archivo. No confundir un acceso directo con una copia independiente.
- Se reservan IDs antes de subir, se reutilizan en reintentos y se rechazan rutas ambiguas con carpetas duplicadas. Los enlaces se guardan con su referencia de evidencia.

Ejemplo de vínculo: recepción → embarque → PNC → SKU → imagen. La evidencia identifica la condición PNC y el registro de cantidades identifica unidades/pallet/posición; el nombre de la carpeta no sustituye estos datos. No borrar carpetas vacías históricas sin contrastar referencias y accesos directos.

9. Autorización Google, correo y catálogo

Google central

El cliente OAuth web central se configura con su origen HTTPS y URI de retorno exactos. El flujo emplea PKCE S256 y estado de uso único. El código incluye el retorno `/api/auth/google/callback`; el valor registrado debe coincidir con la configuración efectiva. Los permisos operativos de Gmail/Drive se distinguen del acceso de usuario al aplicativo.

Integración	Identidad y alcance
Gmail	Cada remitente autoriza su propia cuenta; permisos gmail.readonly y gmail.send.
Drive	Cuenta operativa autorizada por ADMIN; permiso drive. El acceso de ejecutivos/clientes depende de los permisos reales de la carpeta.
Credenciales	Tokens cifrados con Fernet en app_config; clave externa integraciones.key. No se entregan al navegador.
Renovación	El backend renueva access tokens mediante refresh token. Revocación/cambio de cliente exige reconexión de la cuenta.
Compatibilidad	Apps Script se conserva como alternativa anterior; la vía central evita un despliegue individual por supervisor.

Documentos y correo

Los informes se versionan mediante huella y tipo. El usuario selecciona los adjuntos y sus nombres finales. La cola conserva la instantánea de entrega y su resultado. Un envío pendiente no es una confirmación: ante una respuesta interrumpida se consulta el estado y Enviados antes de repetir.

El informe al ejecutivo conserva la estructura original: PALLET/posición con identificadores limpios; el detalle de cantidades y PNC se agrega al final. El informe de auditoría puede incluir mayor desglose. Las cantidades se exportan como valores numéricos; las fotografías incluidas deben tener su vínculo verificado.

Depot y maestro

La API de catálogo/stock se utiliza si las credenciales están habilitadas; existe maestro local y caché. Los archivos CSV/XLSX sirven al flujo operativo de carga y revisión. La integración documentada no implica escritura automática de mercancía en Depot.

10. Configuración, seguridad y sincronización

Parámetro / elemento	Administración
/etc/iv-itsanet.env	Archivo protegido del despliegue; no publicarlo ni reemplazarlo con una plantilla.
VAL_DATABASE_URL	Conexión del motor, host, puerto, esquema y usuario. Mantener la contraseña fuera de documentación y Git.
VAL_DATA_DIR	Ruta persistente para archivos y clave. Valor predeterminado en el código: data/ del repositorio.
VAL_DB_POOL_SIZE / VAL_DB_POOL_RECYCLE	Pool SQL: predeterminados 5 y 3600 segundos. Ajustar con medición de carga.
VAL_SESSION_HOURS	Sesión local: predeterminado 12 horas; revisar valor efectivo.
VAL_COOKIE_SECURE	Cookie segura según HTTPS/configuración; conservar el esquema correcto en el proxy.
DEPOT_*	URL, usuario, clave, rutas y caché; secretos gestionados en el servidor.

- Roles de aplicación: OPERADOR, SUPERVISOR y ADMIN; las autorizaciones se validan en el servidor. ADMIN funcional no equivale a root del sistema operativo.
- La credencial local se conserva como hash PBKDF2-HMAC-SHA256 con sal. La sesión utiliza token aleatorio y cookie HttpOnly / SameSite=Lax.
- La clave externa de integraciones se protege con permisos restringidos. Su pérdida impide recuperar las credenciales cifradas existentes.
- La actualización de vistas combina SSE por recepción con consulta alternativa de 4 segundos. El proxy debe permitir conexiones continuas.
- La presencia informa quién trabaja sobre el SKU; no sustituye los controles de concurrencia e idempotencia. Preservar borradores al actualizar vistas.
- La política institucional define exposición de red, accesos, retención y renovaciones. Los controles descritos no constituyen certificación normativa ni garantía de alta disponibilidad.

11. Calendario de mantenimiento

Las siguientes frecuencias constituyen el plan preventivo recomendado. Su ejecución debe registrarse con fecha, responsable, resultado y acciones pendientes. La programación automática y la retención se verifican en el servidor antes de considerarlas implantadas.

Frecuencia	Actividad	Responsable funcional
Diaria / inicio de jornada	Acceso, pendientes de fotos/correos, errores, catálogo y disponibilidad.	Administrador funcional / supervisor.
Diaria	Verificar respaldo SQL, archivos, clave y configuración; comprobar resultado e integridad.	Administrador de infraestructura.
Semanal	Capacidad de disco/RAM, crecimiento, colas, errores recurrentes y registros del servicio.	Infraestructura + mantenedor.
Mensual	Usuarios/roles, cuentas sin uso, permisos Drive, dependencias y vencimiento de certificado.	Infraestructura + mantenedor + ADMIN.
Trimestral	Ensayo de restauración aislada y revisión de capacidad y continuidad.	Infraestructura + mantenedor.
Antes y después de cada cambio	Pruebas, respaldo, revisión Git, aplicación controlada y verificación funcional.	Mantenedor + infraestructura + supervisor.

Actualización del aplicativo

- Reproducir y corregir en un entorno separado; verificar cantidades, reportes, fotos y permisos afectados. No ejecutar pruebas contra producción.
- Registrar commit completo y ventana del cambio. Usar scripts/actualizar_produccion.sh según su procedimiento; no reinstalar el servicio desde la plantilla servicio.sh.
- El actualizador prepara dependencias antes de la parada y respalda datos/configuración; luego reinicia la aplicación y verifica respuesta.
- Comprobar acceso, una operación representativa, informes y las integraciones que cambien. Conservar borradores/fotos pendientes del navegador.
- Documentar resultado y recuperación prevista. Volver al código anterior no restaura la base de datos.

No actualizar dependencias directamente a la última versión en producción. La evolución V2 se desarrolla aparte de la línea estable: incorporación gradual de otros clientes, revisión de claves SKU, permisos, catálogos, formatos y separación de evidencias antes de cada piloto.

12. Respaldo, recuperación y atención de incidentes

Componente	Contenido necesario
MySQL	Recepciones, eventos, usuarios, configuración, colas, estados e informes.
VAL_DATA_DIR	Archivos operativos retenidos, miniaturas y datos locales persistentes.
Clave y configuración	integraciones.key y entorno protegido, con acceso y copia separados conforme a la política.
Código	Commit compatible, dependencias y configuración del servicio.
Google Drive	Retención, permisos y recuperación de evidencias/documentos externos.

Recomendación: respaldo diario y antes de cada cambio, con copia fuera del servidor y ensayo trimestral. La institución define retención, RPO (pérdida tolerable) y RTO (tiempo de recuperación). El respaldo previo al pase incluido en el código no acredita por sí solo un respaldo diario programado.

- Registrar hora, recepción/SKU, usuario, acción, error y alcance. Preservar evidencia sin incluir credenciales.
- Distinguir red/DNS, Apache, Uvicorn, MySQL y proveedor externo mediante consultas de diagnóstico.
- Ante datos duplicados o cruzados, restringir la operación afectada y revisar eventos; evitar correcciones directas de SQL sin análisis.
- Ante fotos pendientes, conservar el original y reintentar con la identidad registrada. Ante correo pendiente, verificar confirmación y Enviados.
- Restaurar primero en un entorno aislado; validar cantidades, usuarios, archivos y lectura de evidencias. Coordinar cualquier recuperación productiva y su impacto.

13. Consultas de operación y diagnóstico

Ejecutar en la terminal SSH del servidor por personal autorizado. Estas consultas no actualizan ni reinician servicios. Revisar los registros antes de compartirlos y retirar secretos si aparecen.

```
hostname
date -Is
systemctl is-active iv-itsanet apache2 mysql
systemctl show iv-itsanet -p WorkingDirectory -p FragmentPath
sudo ss -lntp '( sport = :3002 or sport = :8443 or sport = :3306 )'
sudo apache2ctl configtest
sudo apache2ctl -S
curl --connect-timeout 5 --max-time 15 -sS -i http://127.0.0.1:3002/health
curl --connect-timeout 5 --max-time 15 -sS -i
https://apps.flexnetuio.com:8443/health
sudo journalctl -u iv-itsanet --since "30 minutes ago" -n 80 --no-pager
uptime
free -h
df -h /
sudo mysql -N -e "SELECT VERSION(), @@version_comment;"
```

Resultado	Área de revisión
Falla acceso directo a 3002	Aplicación, servicio, configuración o dependencia SQL.
3002 responde; HTTPS no responde	Apache, certificado, proxy o conectividad.
Servidor responde; dispositivo no conecta	Red usuaria, DNS, confianza TLS y petición fallida en navegador.
Solo falla Google	OAuth, permisos, cuenta, salida de red o cuota; conservar pendientes.

Consultar la revisión instalada desde el WorkingDirectory efectivo con `git rev-parse HEAD`. El nombre del esquema y las tablas efectivas se obtienen con una sesión SQL autorizada: `SELECT DATABASE();`, `SHOW TABLES;` y `SHOW CREATE TABLE nombre_tabla;`. No publicar la URI de conexión.

14. Diccionario técnico del modelo

Extraído estáticamente de `app/models.py`. Se incluyen campos, tipos declarados, claves y referencias. Los tipos omitidos en mapped_column se infieren de la anotación o de la columna referenciada. CANTIDAD = DECIMAL(14,3), PRECIO = DECIMAL(14,4), FECHA = DATETIME(3) en MySQL y JSON_MAPA = JSON. Las restricciones adicionales se incluyen por tabla.

operadores

Identidad, hash de credencial, rol, habilitación y firma de registros.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
usuario	`String(50)` Mapped[str]	UNIQUE; Índice
nombre	`String(120)` Mapped[str]	—
email	`String(160)` Mapped[str | None]	Índice
pin_hash	`String(200)` Mapped[str]	—
rol	`Enum(Rol)` Mapped[Rol]	—
activo	`Boolean` Mapped[bool]	—
creado_en	`FECHA` Mapped[datetime]	—

sesiones

Sesiones locales y vencimiento; referencia al operador.

Campo	Tipo / anotación	Claves / referencia
token	`String(64)` Mapped[str]	PK
operador_id	`Integer` Mapped[int]	FK → operadores.id; Índice

Campo	Tipo / anotación	Claves / referencia
creada_en	`FECHA` Mapped[datetime]	—
expira_en	`FECHA` Mapped[datetime]	—

productos

Maestro de productos y estado de verificación del catálogo.

Campo	Tipo / anotación	Claves / referencia
sku	`String(80)` Mapped[str]	PK
cliente	`String(60)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
ean	`String(60)` Mapped[str | None]	Índice
unidad	`String(20)` Mapped[str | None]	—
familia	`String(60)` Mapped[str | None]	—
codigo_alternativo	`String(80)` Mapped[str | None]	Índice
inactivo	`Boolean` Mapped[bool]	—
existe_depot	`Boolean` Mapped[bool]	—
origen	`String(20)` Mapped[str]	—
actualizado_en	`FECHA` Mapped[datetime]	—

producto_codigos

Códigos alternativos asociados al SKU del maestro.

Campo	Tipo / anotación	Claves / referencia
codigo	`String(80)` Mapped[str]	PK
valor	`String(80)` Mapped[str]	—
sku	`String(80)` Mapped[str]	FK → productos.sku; Índice
registrado_por	`String(160)` Mapped[str | None]	—
exportar	`Boolean` Mapped[bool]	—
creado_en	`FECHA` Mapped[datetime]	—

recepciones

Cabecera del modelo inicial de recepción; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
codigo	`String(80)` Mapped[str]	UNIQUE; Índice
cliente	`String(60)` Mapped[str]	—
descripcion	`String(300)` Mapped[str | None]	—
estado	`Enum(EstadoRecepcion)` Mapped[EstadoRecepcion]	Índice
archivo_origen	`String(300)` Mapped[str | None]	—
hash_archivo	`String(64)` Mapped[str | None]	—

Campo	Tipo / anotación	Claves / referencia
bultos_declarados	`Integer` Mapped[int | None]	—
creado_por_id	`Integer` Mapped[int | None]	FK → operadores.id
creado_en	`FECHA` Mapped[datetime]	Índice
cerrado_en	`FECHA` Mapped[datetime | None]	—
entregado_en	`FECHA` Mapped[datetime | None]	—

packing_items

Líneas del packing del modelo inicial; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion_id	`Integer` Mapped[int]	FK → recepciones.id; Índice
linea	`Integer` Mapped[int]	—
origen	`String(40)` Mapped[str]	Índice
sku	`String(80)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
referencia_pedido	`String(80)` Mapped[str | None]	—
factura	`String(80)` Mapped[str | None]	—
cantidad	`CANTIDAD` Mapped[float]	—
precio_unit	`PRECIO` Mapped[float | None]	—

Campo	Tipo / anotación	Claves / referencia
precio_total	`PRECIO` Mapped[float | None]	—
num_pieza_reemplazo	`String(80)` Mapped[str | None]	—
embarque	`String(80)` Mapped[str | None]	Índice
extra	`Text` Mapped[str | None]	—

```
(UniqueConstraint('recepcion_id', 'origen', 'linea', name='uq_item_linea'),
Index('ix_item_rec_sku', 'recepcion_id', 'sku'))
```

pallets

Pallets del modelo inicial; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion_id	`Integer` Mapped[int]	FK → recepciones.id; Índice
codigo	`String(40)` Mapped[str]	—
padre	`String(40)` Mapped[str | None]	—
es_subpallet	`Boolean` Mapped[bool]	—
creado_en	`FECHA` Mapped[datetime]	—

```
(UniqueConstraint('recepcion_id', 'codigo', name='uq_pallet_codigo'),)
```

scan_events

Eventos del modelo inicial; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion_id	`Integer` Mapped[int]	FK → recepciones.id; Índice
sku	`String(80)` Mapped[str]	Índice
ean	`String(60)` Mapped[str | None]	—
pallet	`String(40)` Mapped[str]	Índice
posicion_real	`String(60)` Mapped[str | None]	Índice
origen	`String(40)` Mapped[str | None]	Índice
cantidad	`CANTIDAD` Mapped[float]	—
estado_fisico	`Enum(EstadoFisico)` Mapped[EstadoFisico]	—
tipo	`Enum(TipoEvento)` Mapped[TipoEvento]	—
observacion	`String(300)` Mapped[str | None]	—
foto	`String(300)` Mapped[str | None]	—
fotos	`Text` Mapped[str | None]	—
operador_id	`Integer` Mapped[int]	FK → operadores.id; Índice
evento_ref_id	`Integer` Mapped[int | None]	FK → scan_events.id
idempotency_key	`String(80)` Mapped[str | None]	—

Campo	Tipo / anotación	Claves / referencia
dispositivo	`String(80)` Mapped[str | None]	—
creado_en	`FECHA` Mapped[datetime]	Índice

```
(UniqueConstraint('idempotency_key', name='uq_scan_idem'), Index('ix_scan_rec_sku', 'recepcion_id', 'sku'), Index('ix_scan_rec_fecha', 'recepcion_id', 'creado_en'))
```

skus_pendientes

Productos por resolver del modelo inicial; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion_id	`Integer` Mapped[int]	FK → recepciones.id; Índice
sku	`String(80)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
ean	`String(60)` Mapped[str | None]	—
estado_depot	`Enum(EstadoDepot)` Mapped[EstadoDepot]	—
estado	`Enum(EstadoCreacion)` Mapped[EstadoCreacion]	—
candidatos	`Text` Mapped[str | None]	—
notas	`String(300)` Mapped[str | None]	—
detectado_por_id	`Integer` Mapped[int | None]	FK → operadores.id
creado_en	`FECHA` Mapped[datetime]	—

Campo	Tipo / anotación	Claves / referencia
resuelto_en	`FECHA` Mapped[datetime | None]	—

```
(UniqueConstraint('recepcion_id', 'sku', name='uq_pend_rec_sku'),)
```

auditoria

Auditoría del modelo inicial; compatibilidad.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion_id	`Integer` Mapped[int | None]	FK → recepciones.id; Índice
operador_id	`Integer` Mapped[int | None]	FK → operadores.id
accion	`String(60)` Mapped[str]	Índice
detalle	`Text` Mapped[str | None]	—
creado_en	`FECHA` Mapped[datetime]	Índice

app_recepciones

Cabecera operativa, cliente, bultos, estado, fechas, orígenes y embarques.

Campo	Tipo / anotación	Claves / referencia
codigo	`String(60)` Mapped[str]	PK
cliente	`String(60)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
bultos	`Integer` Mapped[int | None]	—

Campo	Tipo / anotación	Claves / referencia
estado	`String(20)` Mapped[str]	Índice
hoja	`String(120)` Mapped[str | None]	—
creado_por	`String(120)` Mapped[str | None]	—
creado_en	`FECHA` Mapped[datetime]	Índice
packing_list_en	`FECHA` Mapped[datetime | None]	—
cerrada_en	`FECHA` Mapped[datetime | None]	—
cerrada_por	`String(120)` Mapped[str | None]	—
totales	`JSON_MAPA` Mapped[dict | None]	—
origenes	`JSON_MAPA` Mapped[list | None]	—
lineas_por_origen	`JSON_MAPA` Mapped[dict | None]	—
formatos	`JSON_MAPA` Mapped[dict | None]	—
embarques	`JSON_MAPA` Mapped[list | None]	—

app_lineas

Líneas originales del packing, orden, SKU, cantidad y columnas adicionales.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
origen	`String(40)` Mapped[str]	—

Campo	Tipo / anotación	Claves / referencia
linea	`Integer` Mapped[int]	—
orden	`Integer` Mapped[int]	—
sku	`String(80)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
cantidad	`CANTIDAD` Mapped[float]	—
extra	`JSON_MAPA` Mapped[dict | None]	—

```
(UniqueConstraint('recepcion', 'origen', 'linea', name='uq_app_linea'),
Index('ix_app_lineas_rec_orden', 'recepcion', 'orden'))
```

app_skus

Proyección de esperado/contado y distribución por pallet y origen.

Campo	Tipo / anotación	Claves / referencia
id	`Integer` Mapped[int]	PK
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
sku	`String(80)` Mapped[str]	Índice
descripcion	`String(300)` Mapped[str | None]	—
esperado	`CANTIDAD` Mapped[float]	—
contado	`CANTIDAD` Mapped[float]	—
verificado	`Boolean` Mapped[bool]	—

Campo	Tipo / anotación	Claves / referencia
estado	`String(20)` Mapped[str]	Índice
en_packing_list	`Boolean` Mapped[bool]	—
tiene_pnc	`Boolean` Mapped[bool]	—
motivo	`String(300)` Mapped[str | None]	—
detectado_por	`String(120)` Mapped[str | None]	—
ultimo_operador	`String(120)` Mapped[str | None]	—
pallets	`JSON_MAPA` Mapped[dict | None]	—
por_origen	`JSON_MAPA` Mapped[dict | None]	—
origenes	`JSON_MAPA` Mapped[list | None]	—
esperado_por_origen	`JSON_MAPA` Mapped[dict | None]	—
creado_en	`FECHA` Mapped[datetime]	—
actualizado_en	`FECHA` Mapped[datetime]	—

```
(UniqueConstraint('recepcion', 'sku', name='uq_app_sku'),)
```

app_calidad_skus

Clasificación y datos de calidad relacionados con el SKU operativo.

Campo	Tipo / anotación	Claves / referencia
sku_id	`Integer` Mapped[int]	PK; FK → app_skus.id; ON DELETE CASCADE

Campo	Tipo / anotación	Claves / referencia
datos	`JSON_MAPA` Mapped[dict]	—

app_presencias

Operador/dispositivo activo y última actividad dentro de una recepción.

Campo	Tipo / anotación	Claves / referencia
recepcion	`String(60)` Mapped[str]	PK; FK → app_recepciones.codigo; ON DELETE CASCADE
dispositivo	`String(64)` Mapped[str]	PK
operador_id	`Integer` Mapped[int]	FK → operadores.id; ON DELETE CASCADE; Índice
sesion_hash	`String(64)` Mapped[str]	Índice
actividad	`String(30)` Mapped[str]	—
visto_en	`FECHA` Mapped[datetime]	Índice

app_informes

Versiones de informe, tipo, huella de datos, estado y autoría.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
clave	`String(80)` Mapped[str]	UNIQUE
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
numero	`Integer` Mapped[int]	—
tipo	`String(20)` Mapped[str]	—

Campo	Tipo / anotación	Claves / referencia
estado	`String(20)` Mapped[str]	—
usuario	`String(160)` Mapped[str]	—
huella	`String(64)` Mapped[str]	—
datos	`JSON_MAPA` Mapped[dict]	—
creado_en	`FECHA` Mapped[datetime]	Índice

```
(UniqueConstraint('recepcion', 'numero', name='uq_app_informe_numero'),)
```

app_auditoria_general

Acciones generales, referencias y datos de auditoría del aplicativo.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
usuario	`String(160)` Mapped[str]	—
accion	`String(40)` Mapped[str]	Índice
referencia	`String(120)` Mapped[str]	Índice
datos	`JSON_MAPA` Mapped[dict]	—
creado_en	`FECHA` Mapped[datetime]	Índice

app_eventos

Libro de eventos operativos: conteos, cero, anulaciones y motivos.

Campo	Tipo / anotación	Claves / referencia
id	`String(120)` Mapped[str]	PK
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
tipo	`String(30)` Mapped[str]	Índice
sku	`String(80)` Mapped[str | None]	Índice
cantidad	`CANTIDAD` Mapped[float]	—
pallet	`String(60)` Mapped[str | None]	—
posicion	`String(60)` Mapped[str | None]	—
origen	`String(40)` Mapped[str | None]	—
usuario	`String(120)` Mapped[str | None]	Índice
anula_a	`String(120)` Mapped[str | None]	Índice
motivo	`String(300)` Mapped[str | None]	—
datos	`JSON_MAPA` Mapped[dict | None]	—
creado_en	`FECHA` Mapped[datetime]	Índice

```
(Index('ix_app_eventos_rec_fecha', 'recepcion', 'creado_en'),)
```

app_huellas

Huellas del packing para reconocer contenido previamente cargado.

Campo	Tipo / anotación	Claves / referencia
id	`String(160)` Mapped[str]	PK
huella	`String(32)` Mapped[str]	Índice
recepcion	`String(60)` Mapped[str]	Índice
codigo	`String(60)` Mapped[str | None]	—
origen	`String(40)` Mapped[str | None]	—
archivo	`String(300)` Mapped[str | None]	Índice
lineas	`Integer` Mapped[int]	—
skus	`Integer` Mapped[int]	—
unidades	`CANTIDAD` Mapped[float]	—
creado_en	`FECHA` Mapped[datetime]	—

app_archivos

Metadatos de archivos, SHA-256, tamaño y referencias confirmadas de Drive.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
tipo	`String(20)` Mapped[str]	—

Campo	Tipo / anotación	Claves / referencia
nombre	`String(200)` Mapped[str]	—
mime	`String(80)` Mapped[str]	—
drive_id	`String(128)` Mapped[str | None]	—
drive_url	`String(300)` Mapped[str | None]	—
sha256	`String(64)` Mapped[str]	—
bytes	`Integer` Mapped[int]	—
usuario	`String(120)` Mapped[str | None]	—
creado_en	`FECHA` Mapped[datetime]	—

```
(UniqueConstraint('recepcion', 'tipo', 'sha256', name='uq_app_archivo_contenido'),)
```

app_archivo_identidades

Secuencia y procedencia de fotografías, independientes del ID remoto.

Campo	Tipo / anotación	Claves / referencia
archivo_id	`String(32)` Mapped[str]	PK; FK → app_archivos.id; ON DELETE CASCADE
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
secuencia	`Integer` Mapped[int]	—
nombre_original	`String(200)` Mapped[str]	—
datos	`JSON_MAPA` Mapped[dict]	—

```
(UniqueConstraint('recepcion', 'secuencia', name='uq_app_foto_secuencia'),)
```

app_archivo_skus

Relación entre un archivo y uno o varios SKU sin duplicar su contenido.

Campo	Tipo / anotación	Claves / referencia
archivo_id	`String(32)` Mapped[str]	PK; FK → app_archivos.id; ON DELETE CASCADE
sku	`String(80)` Mapped[str]	PK; Índice

app_config

Configuraciones, reservas de identidad y credenciales de integración cifradas.

Campo	Tipo / anotación	Claves / referencia
clave	`String(40)` Mapped[str]	PK
datos	`JSON_MAPA` Mapped[dict]	—
usuario	`String(120)` Mapped[str | None]	—
modificado_en	`FECHA` Mapped[datetime]	—

app_entregas

Cola durable de entregas, instantánea de datos, intentos y resultado.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
clave	`String(64)` Mapped[str]	UNIQUE
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
estado	`String(20)` Mapped[str]	Índice

Campo	Tipo / anotación	Claves / referencia
datos	`JSON_MAPA` Mapped[dict]	—
resultado	`JSON_MAPA` Mapped[dict | None]	—
intentos	`Integer` Mapped[int]	—
error	`String(300)` Mapped[str | None]	—
usuario	`String(120)` Mapped[str]	—
proximo_en	`FECHA` Mapped[datetime]	Índice
creado_en	`FECHA` Mapped[datetime]	—

app_ingresos

Datos complementarios del ingreso asociados a la recepción.

Campo	Tipo / anotación	Claves / referencia
recepcion	`String(60)` Mapped[str]	PK; FK → app_recepciones.codigo; ON DELETE CASCADE
datos	`JSON_MAPA` Mapped[dict]	—
actualizado_en	`FECHA` Mapped[datetime]	—

app_unidades_logisticas

Bultos/cajas y relación jerárquica padre-hijo.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice

Campo	Tipo / anotación	Claves / referencia
tipo	`String(20)` Mapped[str]	—
numero	`Integer` Mapped[int]	—
codigo	`String(80)` Mapped[str]	—
padre_id	`String(32)` Mapped[str | None]	FK → app_unidades_logisticas.id; ON DELETE CASCADE; Índice
datos	`JSON_MAPA` Mapped[dict]	—

```
(UniqueConstraint('recepcion', 'tipo', 'numero', name='uq_app_unidad_fisica'),)
```

app_evidencias

Vínculo archivo-recepción-SKU-unidad-categoría y eliminación lógica.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
archivo_id	`String(32)` Mapped[str]	FK → app_archivos.id; ON DELETE CASCADE; Índice
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
unidad_id	`String(32)` Mapped[str | None]	FK → app_unidades_logisticas.id; ON DELETE CASCADE; Índice
sku	`String(80)` Mapped[str | None]	Índice
origen	`String(40)` Mapped[str | None]	—
categoria	`String(20)` Mapped[str]	—
tipo	`String(40)` Mapped[str]	—

Campo	Tipo / anotación	Claves / referencia
protocolo	`String(20)` Mapped[str]	—
usuario	`String(120)` Mapped[str]	—
creado_en	`FECHA` Mapped[datetime]	—
eliminado_en	`FECHA` Mapped[datetime | None]	—

app_copias_drive

Cola y estado de copia a Drive, intentos, próximo intento y error.

Campo	Tipo / anotación	Claves / referencia
archivo_id	`String(32)` Mapped[str]	PK; FK → app_archivos.id; ON DELETE CASCADE
recepcion	`String(60)` Mapped[str]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
estado	`String(20)` Mapped[str]	Índice
datos	`JSON_MAPA` Mapped[dict]	—
usuario	`String(120)` Mapped[str]	—
intentos	`Integer` Mapped[int]	—
proximo_en	`FECHA` Mapped[datetime]	Índice
error	`String(160)` Mapped[str | None]	—

app_stock_depot

Caché de posiciones de stock por SKU y cliente.

Campo	Tipo / anotación	Claves / referencia
sku	`String(80)` Mapped[str]	PK
cliente	`String(60)` Mapped[str]	PK
posiciones	`JSON_MAPA` Mapped[list]	—
actualizado_en	`FECHA` Mapped[datetime]	—

app_impresiones

Trabajos de impresión, estación, reserva y seguimiento de estado.

Campo	Tipo / anotación	Claves / referencia
id	`String(32)` Mapped[str]	PK
clave	`String(80)` Mapped[str]	UNIQUE
estacion	`String(40)` Mapped[str]	Índice
recepcion	`String(60)` Mapped[str | None]	FK → app_recepciones.codigo; ON DELETE CASCADE; Índice
usuario	`String(120)` Mapped[str]	—
estado	`String(20)` Mapped[str]	Índice
datos	`JSON_MAPA` Mapped[dict]	—
reserva	`String(64)` Mapped[str | None]	—
reservado_en	`FECHA` Mapped[datetime | None]	—

Campo	Tipo / anotación	Claves / referencia
creado_en	`FECHA` Mapped[datetime]	—
actualizado_en	`FECHA` Mapped[datetime]	—

15. Referencias y conservación de la documentación

Componente	Archivo de referencia
Arranque / configuración	app/main.py · app/config.py
Conexión / modelo	app/db.py · app/models.py
Archivos / Drive	app/services/archivos.py · drive.py · google_drive_api.py
Clasificación y rutas	app/services/fotos_embarques.py · fotos_identidad.py
Autorización y correo	app/services/google_cuentas.py · google_gmail_api.py · app/routers/google_auth.py
Entregas / permisos	app/services/entregas.py · app/security.py · app/routers/
Mantenimiento	scripts/actualizar_produccion.sh · respaldo_produccion.py · estado_produccion.py
Infraestructura recibida	docs/infraestructura/SERVIDOR_APACHE_MYSQL_20261027.md

Actualizar este manual al cambiar el modelo, las integraciones, la infraestructura o los procedimientos. Conservar el historial documental y la referencia Git de cada liberación. El inventario técnico JSON adjunto permite contrastar tablas, campos y dependencias sin exponer datos operativos ni credenciales.